

Find the Plagiarist

Contents

The plagiarism problem	1
The computational solution: TF-IDF and cosine similarity.....	1
Similarity index.....	1
Input	1
Output	1
References.....	2

The plagiarism problem

Suppose that you have downloaded hundreds or thousands of newspaper articles on a topic. How many of these newspaper articles are basic copies of each other? Is one newspaper typically more involved than others in providing the basic story line to the others? And when newspaper articles are "plagiarized," is the copy exactly the same as the original, or is only a part of the original plagiarized in the copy?

The computational solution: TF-IDF and cosine similarity

The tool answers these questions by measuring how similar the documents in a folder are to one another. It is written in Python and uses **scikit-learn**.

Each document is first turned into a numeric vector using **TF-IDF (Term Frequency – Inverse Document Frequency)**. TF-IDF gives a high weight to words that occur often in a document but rarely across the whole collection — these are the words that best characterize and distinguish that document — and a low weight to words that are common everywhere (e.g., function words). Each document therefore becomes a list of weighted terms; its most heavily weighted terms are its *representative terms*.

The similarity between any two documents is then the **cosine similarity** of their TF-IDF vectors — the cosine of the angle between them. A value of 0 means the two documents share nothing of note; a value of 1 (100%) means they point in the same direction and are, in effect, copies. Because cosine similarity looks at the direction of the vectors rather than their length, a short article and a long one that tell the same story still score as highly similar.

For each document the tool finds the others most similar to it, selects one document as the representative of each cluster of near-duplicates, and reports, for every match, the similarity score and the matching terms (the representative terms the two documents share).

Similarity index

The default threshold for similarity is set at **80%**. Documents that score above this value against the representative document are considered duplicates. This level was arrived at by running several different thresholds on different corpora: lowering it gives too many false positives (documents wrongly classified as similar); raising it may exclude too many genuine duplicates. You can change the threshold in the GUI.

Input

The tool takes in input a folder containing the documents to be compared, in plain-text (.txt) format.

Output

The tool produces:

1. a text file, `document_duplicates.txt`, listing for each file that has duplicates: its representative terms; the name of each duplicate file; the matching score; and the matching terms (the representative terms shared with the duplicate);
2. frequency csv files — `document_similarity_classes_freq.csv`, `document_similarity_classes_time_freq.csv`, `document_similarity_document_instance_classes_freq.csv`, and `document_similarity_Document_classes_freq.csv` — giving the frequency of "plagiarism" by class of involvement (0–10%, 10–20%, 20–30%, ...), by class over time (when the filenames embed dates, aggregated by year, quarter/year, or month/year), and by document/source name (e.g., "The Atlanta Constitution").

These frequencies are displayed as bar and line charts.

References

- Salton, G., & McGill, M. J. (1983). *Introduction to Modern Information Retrieval*. McGraw-Hill. (TF-IDF term weighting.)
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.